Human Development Index (HDI) Predictor

Solution Architecture

Project: Human Development Index (HDI) Predictor
Phase: Project Design Phase

---

1. Architecture Overview

The system follows a 3-layer architecture: User Layer -> Flask Application Layer -> Model Storage & Training Layer.

2. Layer Details

Layer	Components	Technology
User Layer	Web browser (Chrome/Edge/Firefox)	Any modern browser
Presentation Layer	home.html, indexnew.html, resultnew.html	HTML, CSS, Jinja2
Application Layer	app.py (Flask routes, input parsing, prediction logic)	Flask, Python 3
Model Layer	HDI.pkl, country_encoder.pkl, prediction pipeline	Scikit-learn, Pickle, NumPy, Pandas
Data Layer	HDI.csv, EDA plots	Pandas, CSV, Matplotlib/Seaborn

3. Project Folder Structure

```
HDI-Predictor-Project/
├── Flask/                     # Flask web application
│   ├── app.py                 # Routes: /, /Home, /Prediction, /predict
│   ├── HDI.pkl                # Trained Linear Regression model
│   ├── country_encoder.pkl    # LabelEncoder for the Country feature
│   └── templates/             # home.html, indexnew.html, resultnew.html
├── Dataset/
│   ├── HDI.csv                # 144-country dataset
│   └── eda_*.png, actual_vs_predicted.png
├── Training/
│   ├── generate_dataset.py    # Builds Dataset/HDI.csv
│   └── train_model.py         # EDA, preprocessing, training, evaluation, saving
└── README.md
```

4. Entity Relationship Diagram

Key entities in the system:

Entity	Primary Key	Key Attributes
Country	country_id	country_name, region, population
HDI_Input_Data	input_id	user_id (FK), country_id (FK), life_expectancy, mean_years_schooling, gni_per_capita, internet_users
ML_Model	model_id	model_name, algorithm_used, accuracy_score, r2_score, model_file_path
HDI_Prediction	prediction_id	input_id (FK), model_id (FK), predicted_hdi_score, hdi_category, prediction_time
Dataset	dataset_id	dataset_name, source, total_rows, total_columns

5. Relationships

- Country -> HDI_Input_Data: One-to-Many (one country can have multiple input records)
- HDI_Input_Data -> HDI_Prediction: One-to-One (each input record generates one prediction)
- ML_Model -> HDI_Prediction: One-to-Many (one trained model generates multiple predictions)
- Dataset -> ML_Model: One-to-Many (one dataset can train multiple model versions)